Diseño probado de una solución orientada a objetos

Sistema de gestión de envíos nacionales e internacionales

Actividad 06 | Programación Orientada a Objetos II

Escuela Superior de Empresa Ingeniería y Tecnología ESEIT

Integrantes
Jeison Martínez Ramírez
Mayerly Andrea Monroy Malagón
Valerie Pamela Suárez Corredor

Fecha 21 de octubre de 2026

Repositorio remoto [Pegar aquí el enlace después de publicarlo]

Síntesis del caso

LogiTrans es una empresa ficticia que necesita registrar envíos nacionales e internacionales, calcular sus costos y consultar su avance. Cuando las tarifas se calculan en lugares diferentes, pueden aparecer inconsistencias; además, una guía repetida o un cambio de estado incorrecto dificulta el seguimiento. La solución propuesta concentra las reglas verificables en un modelo orientado a objetos y conserva evidencia de su funcionamiento mediante pruebas automatizadas.

El alcance incluye la creación de envíos con datos válidos, la asociación con una ruta y una tarifa, el cálculo monetario en pesos colombianos, el registro de guías únicas y la secuencia de entrega. Los dos tipos de envío comparten un contrato de comportamiento y se procesan a través del supertipo Envio. Cada envío administra sus propios eventos de seguimiento; las rutas y las tarifas pueden compartirse porque son inmutables.

El proyecto se ejecuta en consola con Java 21 y Maven. JUnit 5 verifica escenarios válidos, errores, límites y sustitución polimórfica. La matriz de trazabilidad relaciona reglas, UML, código y pruebas. Dos refactorizaciones se comparan contra una línea base para conservar resultados. Quedan fuera del alcance la autenticación, la base de datos, las tarifas comerciales reales y la integración con transportadoras.

1 Plan técnico y estrategia de trabajo

Se propone un prototipo de consola que permita verificar el dominio antes de añadir persistencia o interfaces gráficas. Las tarifas y recargos son supuestos académicos expresados en COP; no representan precios de una transportadora.

Campo del plan	Definición
1 Problema y alcance	Registrar, cotizar y seguir envíos; se excluyen pagos, usuarios, persistencia y conexión con transportadoras.
2 Reglas de negocio	R1 a R8, con entradas, resultados y excepciones observables. El límite de peso es 50 kg inclusive.
3 Clases y responsabilidades	Siete clases de dominio, un enum de estados y Main como punto de entrada. Cada clase tiene una responsabilidad principal.
4 Relaciones UML	Generalización de Envio; asociaciones con Ruta, Tarifa y RegistroEnvios; composición con EventoSeguimiento.
5 Contrato de jerarquía	Todo subtipo calcula un costo positivo en COP sin modificar estado e historial, y conserva las transiciones comunes.
6 Escenarios de prueba	Positivos, nulos, duplicados, pesos límite, precisión monetaria, estados inválidos y contrato aplicado a ambos subtipos.
7 Trazabilidad	Cada ID aparece en la matriz con elemento UML, método Java, nombre de prueba y resultado.
8 Refactorización	Eliminar cálculo duplicado y separar validación de registro; ejecutar la misma suite en cada etapa.
9 Distribución del equipo	Roles propuestos, revisión cruzada y ramas por intención; ver distribución siguiente.

Distribución propuesta y revisión cruzada

Jeison: implementación y ejecución; Mayerly: reglas y modelo UML; Valerie: pruebas y matriz. Es una propuesta de distribución que el equipo debe validar, no un registro de aportes ya realizados. Cada integrante revisa el producto de otro y puede explicar una regla completa desde el modelo hasta su prueba.

Plan de 220 minutos: análisis y verificación, 50; modelado, implementación, pruebas y refactorización, 140; auditoría y entrega, 30. Se proponen ramas feature/reglas-uml, feature/implementacion y feature/pruebas, con integración después de revisión cruzada. La historia local incluida conserva etapas reales de construcción y refactorización.

2 Requerimientos y criterios de aceptación

ID	Regla verificable y criterio de aceptación
R1	Guía no vacía normalizada con strip; peso mayor que 0 y menor o igual a 50 kg; ruta y tarifa obligatorias. Ambos subtipos rechazan datos inválidos con IllegalArgumentException.
R2	Base y precio por kg positivos con máximo dos decimales. Nacional = base + peso × precio/kg. Internacional añade 45000 COP y arancel. Resultado con dos decimales y HALF_UP una sola vez al final.
R3	La ruta exige origen y destino no vacíos y diferentes al ignorar mayúsculas y espacios exteriores. Se permite compartir una ruta inmutable.
R4	El arancel internacional es obligatorio, mayor o igual a cero y con máximo dos decimales. Cero es válido; un valor negativo o nulo produce IllegalArgumentException.
R5	Cada registro admite una sola guía normalizada. Se distinguen mayúsculas: G-1 y g-1 son guías distintas. Un duplicado o envío nulo produce IllegalArgumentException y no cambia el registro.
R6	Secuencia única CREADO → EN_TRANSITO → ENTREGADO. Saltos, retrocesos, repeticiones y null producen IllegalStateException sin alterar estado ni historial.
R7	Cada envío crea y posee de uno a tres eventos. El exterior no puede insertar eventos ni modificar estructuralmente las listas devueltas; son copias no modificables. Los eventos son inmutables.
R8	El contrato de Envio exige costo positivo, determinista y puro en COP para ambos subtipos. RegistroEnvios suma mediante Envio. Igualdad y hash dependen solo de la guía estable.

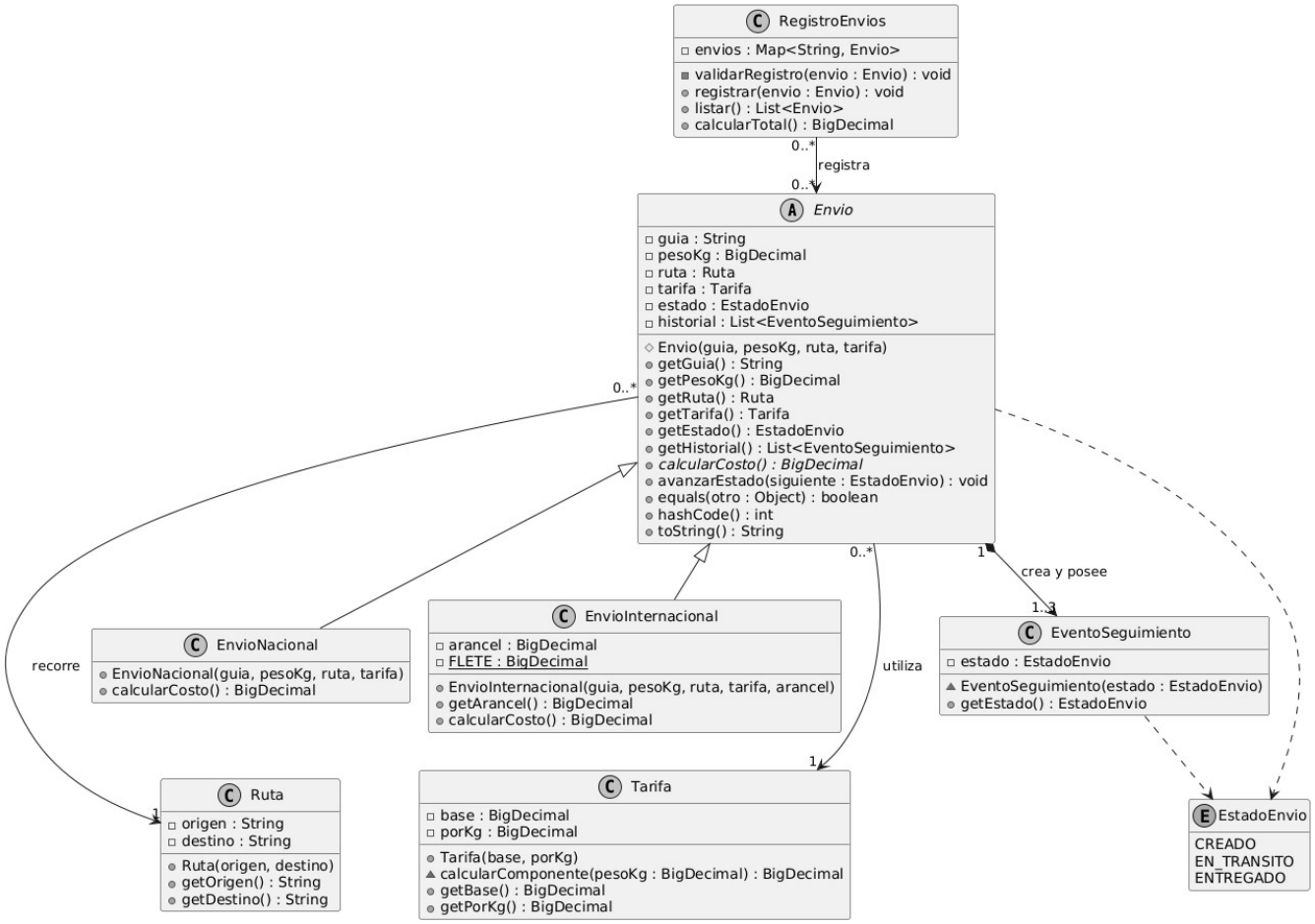
Operación de referencia

Con base de 10000 COP, precio de 2000 COP/kg y peso de 2 kg, el envío nacional cuesta 14000.00 COP. El internacional, con arancel de 5000 COP, cuesta 64000.00 COP. Un registro con ambos totaliza 78000.00 COP. Un registro vacío totaliza 0.00 COP.

El peso no tiene un mínimo positivo fijo: cualquier BigDecimal mayor que cero y hasta 50 es válido. La prueba 0.001 kg verifica un valor pequeño; no pretende representar el menor decimal posible.

3 Modelo UML y relaciones del dominio

Figura 1. Modelo final. Se incluyen siete clases de dominio y el enum EstadoEnvio. Main se excluye del diagrama por ser únicamente el ejecutor de la demostración. El archivo docs/modelo.puml permite editar y regenerar la figura.



Responsabilidades principales

Clase	Responsabilidad
Envio	Validar datos comunes, mantener identidad, estado e historial y definir el contrato de costo.
EnvioNacional	Especializar el cálculo sin recargo internacional.
EnvioInternacional	Validar y aplicar arancel y flete internacional.
Ruta	Conservar origen y destino válidos e inmutables.
Tarifa	Conservar base y precio por kg y calcular el componente común del costo.
RegistroEnvios	Registrar guías únicas, listar y totalizar envíos polimórficamente.
EventoSeguimiento	Representar de forma inmutable un estado alcanzado por un envío.

3 Modelo UML y contrato de sustitución

Justificación de las relaciones

EnvioNacional y EnvioInternacional generalizan Envio porque conservan datos e invariantes comunes y responden al mismo contrato de calcularCosto. Sus fórmulas específicas se resuelven mediante sobrescritura explícita. El cliente RegistroEnvios no pregunta por el tipo concreto ni realiza conversiones.

Cada Envio tiene exactamente una Ruta y una Tarifa; ambas pueden asociarse a cero o muchos envíos. No existe composición con estos objetos: se crean independientemente y son compartibles. La prueba rutaNormalizaEspaciosYPermiteCompartirReferencias confirma que dos envíos pueden usar la misma instancia de Ruta.

RegistroEnvios se asocia con cero o muchos envíos. Un envío puede existir sin registro o participar en varios registros; por eso el extremo RegistroEnvios es 0..*. La unicidad de guía se aplica dentro de cada registro. No se representa composición ni agregación porque no se requiere propiedad sobre los envíos.

Envio compone de uno a tres EventoSeguimiento: crea el evento inicial y agrega uno por transición válida. No acepta eventos externos ni permite transferirlos entre envíos. El constructor del evento tiene visibilidad de paquete, y su clase y atributo son finales. En la API pública, el ciclo de vida pertenece al envío; Java libera objetos cuando dejan de ser alcanzables, sin una operación explícita de destrucción. La propiedad se garantiza frente a clientes externos al paquete, no frente a código interno que viole deliberadamente la convención.

Contrato común y expectativas

Precondiciones de creación: R1 a R3 para ambos subtipos; R4 para el atributo adicional internacional. Sobre un objeto ya válido, calcularCosto no exige parámetros, devuelve siempre un importe positivo con escala 2 y no modifica el estado ni el historial. EnvioInternacional conserva las precondiciones de las operaciones heredadas: su arancel se valida durante la construcción.

La operación avanzarEstado es final, por lo que los subtipos no pueden cambiar sus restricciones. La prueba parametrizada contratoDelSupertipoConservaCostoEstadoEHistorial recibe referencias Envio de ambos tipos, comprueba el costo esperado, repite el cálculo, verifica ausencia de efectos secundarios, recorre la entrega y rechaza una transición posterior.

Correspondencia entre UML y Java

Los signos +, −, # y ~ representan público, privado, protegido y paquete. Los constructores del diagrama omiten tipos de parámetros para conservar legibilidad; los nombres, orden y tipos completos están en el código. Las operaciones son las finales, incluidas las dos extracciones de método. La colección de eventos es privada y se expone como List.copyOf; el mapa de guías solo se modifica mediante registrar. Los getters de referencias retornan objetos inmutables. No se detectaron diferencias de nombres entre el modelo final y el código.

4 Implementación Java

Paquete común: co.edu.eseit.logistica. Los atributos del dominio son privados; los datos de identidad y tarifa son finales. BigDecimal evita aproximaciones binarias en dinero. El redondeo HALF_UP se aplica al resultado final; los importes base, precio por kg y arancel admiten hasta dos decimales.

Cálculo por sobrescritura explícita

```
// EnvioNacional
@Override public BigDecimal calcularCosto() {
    return getTarifa().calcularComponente(getPesoKg())
            .setScale(2, RoundingMode.HALF_UP);
}
// EnvioInternacional
@Override public BigDecimal calcularCosto() {
    return getTarifa().calcularComponente(getPesoKg())
            .add(FLETE).add(arancel).setScale(2, RoundingMode.HALF_UP);
}
```

Cliente que utiliza el supertipo

```
// RegistroEnvios
public BigDecimal calcularTotal() {
    return envios.values().stream().map(Envio::calcularCosto)
            .reduce(new BigDecimal("0.00"), BigDecimal::add);
}
```

Cada llamada a calcularCosto se despacha al subtipo real. El resultado del ejemplo es 14000.00 + 64000.00 = 78000.00 COP. Main presenta ambos envíos y su total en consola, sin entradas externas.

Estado e historial protegidos

```
public final List<EventoSeguimiento> getHistorial() {
    return List.copyOf(historial);
}
public final void avanzarEstado(EstadoEnvio siguiente) {
    boolean valido = (estado == EstadoEnvio.CREADO
            && siguiente == EstadoEnvio.EN_TRANSITO)
        || (estado == EstadoEnvio.EN_TRANSITO
            && siguiente == EstadoEnvio.ENTREGADO);
    if (!valido) throw new IllegalStateException("Transición no permitida");
    estado = siguiente;
    historial.add(new EventoSeguimiento(estado));
}
```

equals y hashCode son finales en Envio y utilizan exclusivamente la guía normalizada, que no cambia. Dos instancias con la misma guía representan la misma identidad, incluso si sus subtipos difieren. El registro rechaza esa duplicidad. toString muestra únicamente guía y estado; no incorpora datos personales.

5 Diseño de pruebas unitarias

La suite LogisticaTest usa JUnit Jupiter 5.11.4. Los nombres expresan reglas observables y las aserciones comparan importes, estados, tamaños e identidad de referencias. No se utilizan servicios externos ni fechas aleatorias. Las pruebas parametrizadas cuentan cada combinación ejecutada como un caso en el reporte de Maven.

ID	Particiones y resultados esperados
R1	Peso −1, 0 y 50.001: rechazo en ambos subtipos. Peso 0.001 y 50: aceptación y costo exacto. Guía null, vacía y espacios: rechazo. Referencias null: rechazo.
R2	Tarifa 0, negativa o 0.001 COP: rechazo. Tarifa 0.01 COP: aceptación. Peso 0.5 con base y precio 0.01 produce 0.015, redondeado a 0.02 COP.
R3	Extremos null, vacíos o espacios: rechazo. Origen y destino equivalentes: rechazo. Ruta compartida y normalizada: aceptación.
R4	Arancel −0.01, −1, 0.001 o null: rechazo. Arancel cero: aceptación y costo internacional 59000.00 COP.
R5	Registro de dos guías diferentes: éxito. Duplicado tras strip o envío null: excepción y conservación del contenido.
R6	Secuencia completa: tres eventos. Salto, repetición, retroceso o null: excepción y mismo tamaño de historial.
R7	clear o add sobre listas públicas: UnsupportedOperationException. Una copia previa conserva su tamaño; eventos de envíos diferentes son instancias diferentes.
R8	Contrato parametrizado para nacional e internacional. Repetición del costo sin efectos secundarios. Total vacío y mixto. Igualdad y hash estables tras cambiar estado.

Prueba del contrato aplicada a cada subtipo

```
@ParameterizedTest @MethodSource("subtipos")
void contratoDelSupertipoConservaCostoEstadoEHistorial(
        Envio envio, BigDecimal esperado) {
    var historial = envio.getHistorial();
    assertAll(() -> assertEquals(esperado, envio.calcularCosto()),
        () -> assertEquals(esperado, envio.calcularCosto()),
        () -> assertTrue(envio.calcularCosto().signum() > 0),
        () -> assertEquals(EstadoEnvio.CREADO, envio.getEstado()),
        () -> assertEquals(historial, envio.getHistorial()));
    // El archivo completo verifica avance y rechazo después de ENTREGADO.
}
```

subtipos suministra un EnvioNacional con esperado 14000.00 y un EnvioInternacional con esperado 64000.00. Así se verifica el contrato desde el supertipo con resultados concretos de cada implementación.

5 Evidencias de ejecución y sensibilidad a defectos

Entorno verificado: Amazon Corretto OpenJDK 21.0.12; Maven 3.9.9; JUnit Jupiter 5.11.4; Linux x86_64. Comando de validación: mvn -B -ntp clean test. Se ejecutó la misma suite en la línea base y después de cada refactorización.

Etapa	Casos	Fallos	Errores	Omitidos	Resultado
Línea base	31	0	0	0	BUILD SUCCESS
Refactorización 1	31	0	0	0	BUILD SUCCESS
Refactorización 2	31	0	0	0	BUILD SUCCESS
Revisión limpia final	31	0	0	0	BUILD SUCCESS

Reportes completos: evidencias/01_linea_base.txt, 02_refactor_tarifa.txt, 03_refactor_registro.txt y 05_revision_limpia.txt. Los XML JUnit de cada etapa también se incluyen. Maven genera nuevos reportes en target/surefire-reports al ejecutar el proyecto; target no se conserva en el repositorio.

Extracto del reporte final

```
Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Comprobación deliberada de un defecto

Se sustituyó temporalmente el flete de 45000 por 45001 COP. La suite detectó diferencias monetarias y terminó con BUILD FAILURE. Se restauró el valor original antes de la revisión final. Esta comprobación demuestra sensibilidad a ese defecto concreto; no equivale a un análisis exhaustivo de mutaciones.

```
Flete modificado temporalmente: 45000 -> 45001 COP
Tests run: 31, Failures: 7, Errors: 0, Skipped: 0
BUILD FAILURE
Valor original restaurado y revisión limpia aprobada.
```

El registro completo de esta ejecución negativa se conserva en evidencias/04_mutacion_detectada.txt. El parche temporal está en evidencias/mutacion_flete.diff. La versión final no contiene el defecto.

Salida de la demostración

```
COL-101 CREADO COP 14000.00
INT-901 CREADO COP 64000.00
Total COP 78000.00
```

La revisión limpia recompiló desde cero después de restaurar el defecto. Los resultados documentados proceden de las ejecuciones incluidas; no se presentan porcentajes de cobertura porque no se realizó medición de cobertura.

6 Matriz de trazabilidad

Todas las reglas cuentan con implementación y pruebas identificables. Los nombres siguientes son métodos de LogisticaTest; su resultado en la revisión final es aprobado. Una prueba parametrizada puede producir varios casos ejecutados.

ID	Elemento UML y Java	Prueba JUnit	Resultado
R1	Envio y subtipos Envio constructor	rechazaPesoFueraDelIntervaloEnAmbosSubtipos; aceptaPesosValidos CercanosALosLimites; rechazaGuiaAusenteEnAmbosSubtipos; rechaza ReferenciasObligatoriasNulasEnAmbosSubtipos	Aprobado
R2	Tarifa y subtipos Tarifa constructor y calcularComponente; calcularCosto	rechazaTarifasNoPositivasOPrecisionExcesiva; rechazaTarifasNulas; aceptaTarifaMinimaYRedondeaUnaSolaVez; contratoDelSupertipo ConservaCostoEstadoEHistorial	Aprobado
R3	Asociación Envio a Ruta Ruta constructor	rutaNormalizaEspaciosYPermiteCompartirReferencias; rechaza ExtremosDeRutaAusentes; rechazaRutaConOrigenYDestino Equivalentes	Aprobado
R4	EnvioInternacional Constructor y calcularCosto	rechazaArancelNegativoOPrecisionExcesiva; aceptaArancelCero YRechazaNulo	Aprobado
R5	RegistroEnvios a Envio registrar y validarRegistro	registraEnviosDistintosYRechazaGuiaDuplicadaSinAlterarRegistro; rechazaRegistroNuloYConservaRegistroVacio	Aprobado
R6	Envio y EstadoEnvio avanzarEstado	rechazaSaltosRepeticionesRetrocesosYNulosSinAlterarHistorial; contratoDelSupertipoConservaCostoEstadoEHistorial	Aprobado
R7	Composición Envio a EventoSeguimiento getHistorial; avanzarEstado; listar	protegeColeccionesYPropiedadDelHistorial	Aprobado
R8	Jerarquía Envio y RegistroEnvios calcularCosto; calcularTotal; equals; hashCode; toString	contratoDelSupertipoConservaCostoEstadoEHistorial; igualdadYHash PermanecenEstablesAlCambiarEstado; registroVacioYMixtoCalculan TotalMedianteSupertipo	Aprobado

La matriz editable completa, sin cortes visuales en los identificadores, se encuentra en docs/trazabilidad.csv. Los mismos ID R1 a R8 se mantienen en el informe y el README.

7 Primera refactorización

Señal de diseño y alternativa elegida

El cálculo base + peso × precio por kg aparecía en ambos subtipos. Mantener la duplicación implicaba modificar dos clases ante un cambio de la fórmula común. Se seleccionó Extraer método y mover el comportamiento común a Tarifa, que ya contiene los valores necesarios. Se mantuvo la especialización por sobrescritura para el recargo internacional.

Antes de extraer el componente común

```
// EnvioNacional
return getTarifa().getBase().add(getPesoKg().multiply(getTarifa().getPorKg()))
        .setScale(2, RoundingMode.HALF_UP);
// EnvioInternacional
return getTarifa().getBase().add(getPesoKg().multiply(getTarifa().getPorKg()))
        .add(FLETE).add(arancel).setScale(2, RoundingMode.HALF_UP);
```

Después de extraer el componente común

```
// Tarifa
BigDecimal calcularComponente(BigDecimal pesoKg) {
    return base.add(pesoKg.multiply(porKg));
}
// EnvioNacional
return getTarifa().calcularComponente(getPesoKg())
        .setScale(2, RoundingMode.HALF_UP);
// EnvioInternacional
return getTarifa().calcularComponente(getPesoKg())
        .add(FLETE).add(arancel).setScale(2, RoundingMode.HALF_UP);
```

Cambio UML: Tarifa incorpora ~calcularComponente(pesoKg : BigDecimal) : BigDecimal. Se mantiene la asociación Envio–Tarifa y la jerarquía. El método tiene visibilidad de paquete porque recibe un peso ya validado por Envio y no constituye una API pública independiente.

Beneficio: una sola definición de la fórmula común y responsabilidad monetaria cohesionada. Se preservó el redondeo al final; redondear antes de añadir recargos sería una decisión diferente y no formó parte de este cambio.

Evidencia: 31 casos aprobados antes y 31 después, con cero fallos, errores u omisiones. Las pruebas del contrato, los pesos límite y el redondeo protegen R1, R2 y R8. Comparar evidencias/01_linea_base.txt con 02_refactor_tarifa.txt y revisar docs/refactor_1.diff.

7 Segunda refactorización

Señal de diseño y alternativa elegida

registrar mezclaba las condiciones de admisión con la inserción en el mapa. Aunque el método era corto, reunía dos intenciones: verificar unicidad y modificar el registro. Se eligió Extraer método para hacer visible la validación antes de la escritura, conservando el orden de comprobación y los mensajes de excepción.

Antes de separar la validación

```
public void registrar(Envio envio) {
    if (envio == null) throw new IllegalArgumentException("Envío obligatorio");
    if (envios.containsKey(envio.getGuia())) {
        throw new IllegalArgumentException("Guía duplicada: " + envio.getGuia());
    }
    envios.put(envio.getGuia(), envio);
}
```

Después de separar la validación

```
public void registrar(Envio envio) {
    validarRegistro(envio);
    envios.put(envio.getGuia(), envio);
}
private void validarRegistro(Envio envio) {
    if (envio == null) throw new IllegalArgumentException("Envío obligatorio");
    if (envios.containsKey(envio.getGuia())) {
        throw new IllegalArgumentException("Guía duplicada: " + envio.getGuia());
    }
}
```

Cambio UML: RegistroEnvios incorpora −validarRegistro(envio : Envio) : void. La asociación y las multiplicidades no cambian. Las pruebas no invocan el método privado: verifican el efecto observable de registrar, lo que evita acoplar la suite a la extracción.

Beneficio: la operación pública expresa la secuencia validar y registrar; las reglas de admisión quedan localizadas. La inserción sucede únicamente después de validar. La mejora es de legibilidad, no de rendimiento ni de concurrencia.

Evidencia: 31 casos aprobados antes y 31 después. registraEnviosDistintosYRechazaGuiaDuplicadaSinAlterarRegistro y rechazaRegistroNuloYConservaRegistroVacio protegen la aceptación, la unicidad y la ausencia de efectos ante errores. Los reportes 02_refactor_tarifa.txt y 03_refactor_registro.txt permiten comparar el resultado. El cambio completo se conserva en docs/refactor_2.diff.

Los archivos docs/modelo_antes.puml y docs/modelo.puml permiten comparar las operaciones de las dos versiones; docs/modelo_intermedio.puml corresponde a la primera extracción. Los commits separados documentan la intención de cada cambio sin mezclar funcionalidades nuevas.

8 Discusión de diseño y cierre

La herencia se utiliza para representar dos formas de envío que comparten identidad, peso, ruta, tarifa y seguimiento. La decisión se sostiene en el contrato de comportamiento: ambos subtipos calculan un costo positivo en pesos colombianos sin modificar su estado y admiten la misma secuencia de entrega. El registro trabaja con referencias Envio, por lo que puede totalizar una colección mixta sin conocer las clases concretas. Las pruebas de contrato verifican esta sustitución con resultados monetarios específicos.

La composición se reserva para los eventos de seguimiento. Cada envío crea sus eventos, conserva su historial y no ofrece operaciones para incorporar objetos externos. Las rutas y las tarifas, en cambio, son asociaciones compartibles: sus instancias son inmutables y no dependen de un envío particular. Esta distinción evita atribuir propiedad fuerte a elementos que tienen existencia independiente. La visibilidad privada de los atributos y las copias no modificables impiden que un cliente elimine eventos o inserte envíos eludiendo las validaciones del registro.

La estrategia de pruebas prioriza reglas observables. Se comprobaron entradas válidas, nulos, duplicados, límites de peso, precisión monetaria y transiciones inválidas. Los nombres describen la condición o el comportamiento esperado. El cambio deliberado del flete demostró que la suite detecta al menos una alteración concreta de la fórmula; no permite afirmar que todos los defectos posibles estén cubiertos. Las refactorizaciones mantuvieron las pruebas y sus expectativas: primero se eliminó la fórmula duplicada y luego se separó la validación del registro. Los reportes antes y después respaldan la conservación del comportamiento.

La solución tiene limitaciones. El registro está en memoria y no es seguro para accesos concurrentes; dos registros independientes pueden contener una misma guía. El historial representa estados, pero no registra fechas ni responsables. Las rutas no verifican fronteras y los importes internacionales son supuestos académicos. Como mejora futura, se propone persistencia transaccional con unicidad global de guía y una política de recargos configurable. Estas extensiones deberían añadirse con nuevas reglas y pruebas, sin presentarlas como una refactorización que conserva exactamente el alcance actual.

Auditoría de coherencia

Se cotejaron los nombres de clases, operaciones y paquete; las asociaciones de una ruta y una tarifa se reflejan en validaciones no nulas. El máximo de tres eventos deriva de las dos transiciones posibles. Las ocho reglas tienen pruebas en la matriz. Las dos extracciones están presentes en el UML final. La revisión limpia reconstruyó la suite y la demostración con resultados esperados.

Defensa oral sugerida

¿Por qué herencia? Porque ambos tipos satisfacen el contrato de Envio, comprobado por la prueba parametrizada. ¿Por qué composición? Porque los eventos se crean dentro de su envío y no pueden transferirse mediante la API pública. ¿Qué protege una refactorización? La misma suite ejecutada antes y después, junto con el contraste del código y del diagrama.

9 Reproducibilidad y referencias

Estructura del proyecto y ejecución

```
ACT06_Logistica/
  pom.xml
  README.md
  src/main/java/co/edu/eseit/logistica/
  src/test/java/co/edu/eseit/logistica/LogisticaTest.java
  docs/          UML, trazabilidad y diferencias de refactorización
  evidencias/    reportes de cada etapa y revisión final
```

Requisitos: JDK 21, Maven 3.9.9 e internet para la primera descarga de dependencias. Desde la carpeta que contiene pom.xml, comprobar java -version y mvn -version; luego ejecutar:

```
mvn -B -ntp clean test
java -cp target/classes co.edu.eseit.logistica.Main
```

El archivo README contiene instrucciones para IntelliJ o Eclipse, recuperación de la historia Git desde un bundle y publicación en GitHub. La dirección remota debe incorporarse en la portada antes de entregar. El repositorio local no se presenta como un enlace público ya existente.

Referencias

Escuela Superior de Empresa Ingeniería y Tecnología. (s. f.-a). Guía didáctica de la actividad 06: Diseño probado de una solución orientada a objetos [Guía de curso].

Escuela Superior de Empresa Ingeniería y Tecnología. (s. f.-b). Modelado UML, pruebas unitarias y refactorización de jerarquías (TG-04, pp. 28–33) [Material de curso].

JUnit Team. (2024). JUnit 5 user guide (Version 5.11.4). https://docs.junit.org/5.11.4/user-guide/

Object Management Group. (2017). Unified Modeling Language (Version 2.5.1). https://www.omg.org/spec/UML/2.5.1

Oracle. (2023). The Java language specification: Java SE 21 edition (Chapter 8, Classes). https://docs.oracle.com/javase/specs/jls/se21/html/jls-8.html

El diseño retoma la cadena reglas–modelo–código–pruebas y la revisión de refactorizaciones descritas en el TG-04 (ESEIT, s. f.-b, pp. 28–31). La notación se apoya en OMG (2017); la generalización y sobrescritura, en Oracle (2023); las pruebas parametrizadas y aserciones, en JUnit Team (2024).